

Documentação Técnica

Pipeline de Análise Dinâmica e Feedback Formativo

Classificador de Erros de Memória em C — CodeBench

PIBIC — Universidade Federal do Amazonas (UFAM)

Guilherme Pereira

Detalhamento dos arquivos-chave, arquitetura e fluxo de informação

1. Visão geral

Este projeto é um **pipeline determinístico de análise dinâmica** que recebe código C submetido ao juiz online **CodeBench**, executa esse código sob ferramentas de instrumentação de memória, identifica o erro de forma automática e produz duas coisas: um **diagnóstico técnico** (tipo do erro, CWE, linhas envolvidas) e um **feedback formativo** que orienta o aluno a corrigir sozinho — sem entregar a solução pronta.

O princípio que organiza todo o sistema é a **agência local**: o fluxo de decisão pertence a regras determinísticas (as ferramentas de análise e tabelas de mapeamento), não à Inteligência Artificial. O modelo de linguagem (LLM) não decide o diagnóstico; ele apenas *interpreta* o resultado já apurado para conversar com o aluno. Isso traz três benefícios diretos: o diagnóstico é **rápido** (não espera o LLM), **estável** (a mesma entrada gera sempre a mesma classificação) e **auditável** (cada rótulo tem origem rastreável no log da ferramenta). Também reduz o risco de alucinação de um modelo pequeno (7B parâmetros).

O que o pipeline detecta

Erros de memória em C capturados por instrumentação em tempo de execução: buffer overflow (heap e stack), use-after-free, double-free e free inválido, uso de valor não inicializado, memory leak, null-pointer dereference (SIGSEGV) e erros de compilação. Cada erro é mapeado para um **CWE** (Common Weakness Enumeration, catálogo do MITRE que identifica fraquezas de software com um código citável, ex.: CWE-416 = Use After Free).

2. Arquitetura em cascata (fail-fast)

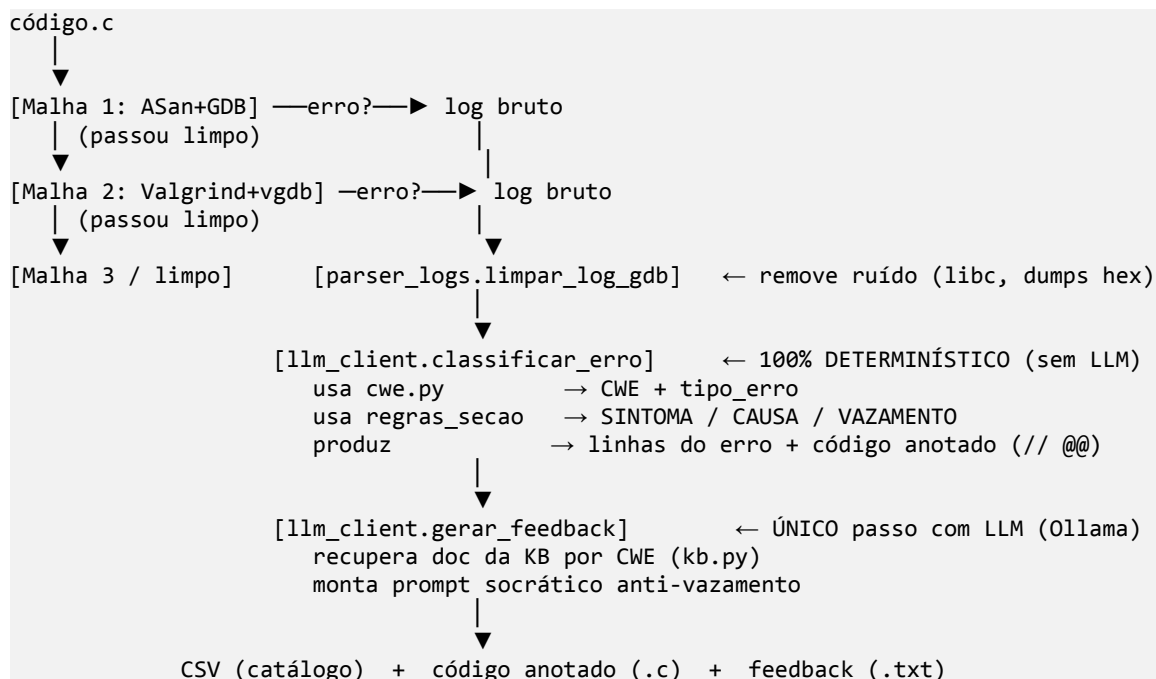
O código do aluno passa por **malhas** de análise em sequência. A primeira malha que encontra um erro interrompe o fluxo — é o comportamento *fail-fast*. Cada malha cobre uma classe de erro que a outra não cobre bem:

- **Malha 1 — AddressSanitizer + GDB**: erros espaciais de acesso à memória (overflows, use-after-free) e crashes por sinal (SIGSEGV etc.), com o GDB capturando o estado exato da falha.
- **Malha 2 — Valgrind (Memcheck) + vgdb**: só executa se a Malha 1 passou limpa. Detecta valores não inicializados e memory leaks, com o GDB conectado ao vivo (vgdb) no ponto exato do erro.
- **Malha 3 — Perfilador de complexidade**: exploratória e desativada por padrão (MALHA_3_ATIVA=False). Infere a complexidade Big-O por regressão sobre entradas de tamanho crescente.

Decisão de projeto importante: os **memory leaks são delegados à Malha 2**. Na Malha 1, o LeakSanitizer é desligado (detect_leaks=0), pois ele usa ptrace — o mesmo mecanismo do GDB — e os dois entram em conflito. Além disso, o Valgrind+vgdb produz um relatório de leak mais rico, cobrindo leaks indiretos (ex.: nós internos de uma lista encadeada).

3. Fluxo de informação

No modo catálogo (o `orquestrador.py`), cada submissão percorre o caminho abaixo:



O ponto central: o **diagnóstico sai inteiro das ferramentas**. O LLM entra apenas no fim, para transformar esse diagnóstico em um texto que leva o aluno a descobrir a correção.

3.1. Anotação inline (// @@) — inspirada no FLAME

Em vez de dizer ao modelo "o erro está na linha 23", o pipeline **anota o próprio código** com um comentário na linha exata reportada pela ferramenta:

```
raiz->esq = novo; // @@ [CAUSA] o bloco de memória foi LIBERADO (free) aqui
...
printf("%d", raiz->esq->n); // @@ [SINTOMA] uso de memória já liberada
```

O artigo *FLAME* demonstra, por ablação, que anotar a linha no código supera de longe passar o número da linha em texto (a acurácia top-1 cai de 62,7% para 15,8% quando se usa referência numérica), porque LLMs têm baixa "compreensão numérica" e se perdem em contextos longos. A vantagem deste pipeline sobre o FLAME: lá é a própria LLM que anota e a marcação desalinha ao reformatar o código (exigindo fuzzy matching por similaridade); aqui quem anota é o pipeline, ancorado na linha exata da ferramenta — logo a anotação é **determinística** e dispensa o fuzzy matching.

4. Duas faces do mesmo pipeline: catálogo e API

- **Modo catálogo (offline):** o `orquestrador.py` varre uma pasta de submissões e gera um CSV com o diagnóstico e o feedback de cada uma. É o modo de pesquisa e avaliação em lote.
- **Modo API (online):** a pasta `api/` expõe o mesmo pipeline como um serviço FastAPI para o CodeBench. A detecção roda isolada num contêiner Docker (porque o código do aluno é não confiável) e o LLM roda no host (o contêiner não tem rede). Dois endpoints: `/analisar` (JSON, diagnóstico imediato) e `/feedback` (streaming SSE, o texto "sendo digitado").

5. Detalhamento dos arquivos — núcleo do pipeline (src/)

5.1. src/config.py — configuração central

Concentra todas as constantes do pipeline num único lugar; todos os módulos importam daqui. Centralizar evita valores espalhados e permite mudar o comportamento sem caçar constantes pelo código.

Constante	Papel
PASTA_CODIGOS	Pasta das submissões a analisar.
ARQUIVO_CSV_SAIDA	Caminho do catálogo CSV de saída.
PASTA_ANOTADOS / PASTA_FEEDBACKS	Onde salvar códigos anotados e feedbacks .txt.
PASTA_KB	Base de conhecimento (um documento por CWE).
URL_LLM_LOCAL / NOME_MODELO	Endereço do Ollama e modelo (qwen2.5-coder:7b).
LLM_TIMEOUT_S / LLM_KEEP_ALIVE	Timeout generoso (cold start) e manutenção do modelo em memória.
TEMPERATURA_CLASSIFICACAO / _FEEDBACK	0.0 (determinística) e 0.7 (texto mais natural).
MODULO_ANALISADO	Variável de ablação: inline / numerica / nenhuma.
MALHA_3_ATIVA	Liga/desliga o perfilador (Malha 3).
VALGRIND_TIMEOUT_S / VALGRIND_POLL_S	Teto e intervalo de verificação da Malha 2.

5.2. src/malha1_asan.py — Malha 1 (ASan + GDB)

Função principal: `executar_malha1_asan(caminho_codigo, binario_saida, entrada)`. Trabalha em quatro fases:

- **Compilação** com gcc e flags de instrumentação: `-fsanitize=address` (injeta "redzones" que abortam em acesso fora dos limites), `-g` (símbolos de depuração), `-fsanitize-address-use-after-scope` e `-fno-omit-frame-pointer` (backtraces fiéis).
- **Padrão da linguagem fixado em `-std=gnu11`**: sem isso, um GCC recente usa C23, onde `false/true/bool` viraram palavras-chave e quebram códigos legados que os redefinem — gerando falsos positivos que não ocorrem no juiz.
- **Ambiente**: `ASAN_OPTIONS=abort_on_error=1:detect_leaks=0` — aborta no 1º erro (para o GDB capturar o estado) e desliga o LeakSanitizer (leaks vão para a Malha 2).
- **Execução via GDB** em modo `--batch` com `bt full` (backtrace completo com variáveis locais). Se o erro escapa como sinal cru (SIGSEGV), o GDB o intercepta antes do ASan; por isso a função reconhece uma lista de sinais fatais (`_SINAIS_FATAIS`) para não deixar o crash passar sem diagnóstico.

Retorna um dicionário com tipo (`asan / crash / compilacao`) e o log bruto, ou `None` se o código passou limpo (segue para a Malha 2).

5.3. src/malha2_valgrind.py — Malha 2 (Valgrind + vgdb)

Função principal: `executar_malha2_valgrind(...)`. Roda o programa sob o Memcheck com o GDB conectado ao vivo pelo vgdb, capturando não-inicializados e leaks.

Sincronização por evento (o ponto delicado)

Uma versão anterior lançava o Valgrind e fazia um `time.sleep(1.5)` fixo antes de conectar o GDB — um anti-padrão: a espera fixa entra em corrida com o startup do Valgrind. Se o startup demora mais que isso, o GDB conecta com o programa ainda carregando a libc e mata o processo antes do main, gerando log inútil.

A solução é esperar por um **evento real**, com teto de segurança, na função `_aguardar_evento_valgrind`, que retorna um de três estados:

- `erro_execucao` — o Valgrind congelou num erro (marcador no stderr) e espera o vgdb; conecta-se o GDB sem corrida.
- `terminou` — o processo encerrou sozinho; lê-se o relatório de leak-check.
- `timeout` — estourou o teto (ex.: loop infinito).

Flags de origem elevam a qualidade do diagnóstico: `--track-origins=yes` (onde o valor não inicializado nasceu), `--keep-stacktraces=alloc-and-free` (guarda alocação E liberação, para `use-after-free`) e `--read-var-info=yes` (nome e linha de declaração da variável). O stderr é drenado numa thread em segundo plano para poder ser inspecionado enquanto o processo roda.

5.4. `src/deteccao_entrada.py` — geração do stdin

Compartilhado pelas Malhas 1 e 2. Fornece o stdin que exercita o código, evitando que o processo trave esperando entrada do terminal. Ordem de prioridade:

- Caso de teste real por arquivo (`<nome>.in` ao lado do `.c`).
- Caso de teste compartilhado pela pasta (`_entrada.in`) — útil quando a pasta reúne várias submissões do mesmo exercício.
- **Heurística de escala**: sem `.in`, detecta um `scanf("%d", &var)` cuja variável controla um laço/alocação e sintetiza uma entrada mínima (ex.: `"5\n1 2 3 4 5\n"`).
- Nenhuma: entrada fixa não reconhecida — herda o stdin do ambiente.

5.5. `src/parser_logs.py` — limpeza do log

Função `limpar_log_gdb(log_bruto, nome_arquivo)`. Um backtrace cru pode ter mais de 200 linhas de ruído (frames da libc, dumps hexadecimais, shadow bytes do ASan). Esta função filtra e preserva só o que importa para o diagnóstico, por uma sequência de filtros: uma "guilhotina" que para no mapa de shadow bytes, descarte de linhas muito longas (dumps), rastreio de fronteiras de frame (para manter só frames do código do aluno), lista de alertas críticos (tokens que identificam o tipo do erro) e captura de linhas de localização/contexto de memória e de variáveis locais do aluno.

5.6. `src/cwe.py` — mapeamento determinístico para CWE

Traduz o relatório da ferramenta em um código CWE por busca de expressões regulares — nunca por consulta ao LLM. Fundamenta-se no catálogo do MITRE e no Juliet Test Suite (NIST SARD).

- `REGRAS_CWE`: lista de (regex, CWE, nome) em ordem de prioridade. A ordem desambigua erros que compartilham marcadores no log — ex.: `double-free` e `use-after-free` ambos citam `"free'd"`, então as regras de liberação vêm antes.
- `classificar_cwe(log)`: retorna (cwe_id, cwe_nome), ou ("-", "-") se nada casar.
- `classificar_cwe_tipo(log)`: além do CWE, devolve o nome técnico do erro (tipo_erro) via o mapa `_TIPO_POR_CWE`. Este rótulo passou a ser derivado do CWE de forma determinística, em vez de pedido ao LLM (era redundante).

5.7. `src/regras_secao.py` — regras de seção do relatório

Concentra num único lugar o "contrato" com o texto que ASan e Valgrind imprimem. A tabela `REGRAS_SECAO` associa cada trecho do log a um papel — SINTOMA (onde o erro se manifesta), CAUSA (onde nasce), VAZAMENTO (a alocação nunca liberada) e ORIGEM (onde o bloco foi alocado). É o que permite anotar

sintoma e causa separadamente. Ajustar uma classe de erro (ex.: nova versão do Valgrind muda um texto) se faz editando só uma linha aqui.

5.8. `src/kb.py` — base de conhecimento por chave exata

Função `recuperar_kb(cwe_id)`: dado o CWE já determinado, lê o documento curado em `base_conhecimento/<CWE-ID>.md`. Usa **chave exata** em vez de busca semântica (embeddings) de propósito: o CWE é uma chave estruturada e confiável, então recuperar por chave elimina, por construção, o "distrator" (passagens parecidas-porém-erradas que pioram a geração de um modelo pequeno). Não precisa de vector DB, é auditável e a "busca" é apenas uma leitura de arquivo. Valida o formato do id (proteção contra path traversal) e, se não há documento, retorna vazio (degradação graciosa).

6. src/llm_client.py — classificação, anotação e feedback

É o arquivo mais extenso do núcleo. Reúne três responsabilidades: extrair as linhas do erro e anotar o código (determinístico), classificar o erro (determinístico) e gerar o feedback (único passo com LLM). Os principais métodos:

`_chamar_llm(prompt, tentativa, temperatura)`

Camada única de comunicação com o Ollama. Faz a requisição HTTP em streaming, com format=json e a temperatura recebida por fase. Mantém o modelo carregado entre chamadas (keep_alive) para evitar cold start.

`_extrair_pontos_anotacao(log, nome_arquivo)`

Varre o log (de preferência o bruto) e extrai, de forma determinística, os pontos que a ferramenta reportou, associando cada um a um papel (SINTOMA/CAUSA/VAZAMENTO). Usa duas expressões regulares: uma para "arquivo:linha" e outra que distingue o frame do GDB do frame do Valgrind pelo formato do "at", dando prioridade ao relatório estruturado. Isso é essencial para o caso do SIGSEGV puro (null-deref), em que o ASan não gera relatório e a linha só existe no backtrace do GDB.

`_anotar_codigo(codigo_fonte, pontos)`

Injeta os comentários // @@ [PAPEL] no fim das linhas exatas. Quando sintoma e causa caem na mesma linha, combina os marcadores em vez de sobrescrever.

`_fim_da_funcao(linhas, linha_abertura)`

Encontra a chave } que fecha uma função por casamento de chaves, ignorando chaves dentro de strings, chars e comentários. É o limite superior robusto de varredura (substituiu uma antiga janela fixa de 40 linhas), garantindo que a busca nunca cruze para a função seguinte.

`_refinar_causa_uninit(pontos, codigo_fonte)`

Para valores não inicializados, o Valgrind reporta a origem na linha de abertura da função (o frame é alocado de uma vez), não na declaração da variável. Este método move a marca da CAUSA para a linha da declaração sem inicializador — o ponto pedagogicamente correto —, mas só quando isso é inequívoco (exatamente uma declaração candidata entre a função e o uso). Caso contrário, mantém a linha original: uma trava de segurança que nunca piora o que já existe.

`_evidencia_do_log(log)`

Escolhe deterministicamente uma linha real do relatório para servir de evidência (ex.: "...definitely lost"). É a rede de segurança do campo evidencia_log: se algo tentasse copiar a anotação // @@ (que é rótulo do pipeline, não prova), este método a substitui por uma linha verdadeira do log.

`classificar_erro(log_limpo, codigo_fonte, nome_arquivo, log_bruto)`

A 1ª etapa de análise, hoje **100% determinística (sem LLM)**. Junta os passos acima: extrai os pontos, refina a causa de não-inicializado, anota o código, calcula CWE + tipo_erro (via cwe.py) e a evidência do log. Retorna o dicionário com tipo_erro, cwe_id, cwe_nome, linha_sintoma, linha_causa, evidencia_log e codigo_annotado. Como não chama o LLM, o diagnóstico é imediato.

`montar_prompt_feedback(codigo, analise, saida)`

Fonte única das regras pedagógicas do feedback, usada por dois caminhos (JSON no modo catálogo, texto puro no streaming da API) para as regras nunca divergirem. Recupera o documento da KB pelo CWE, injeta o diagnóstico e monta um prompt socrático com regras anti-vazamento: não revelar a correção, basear-se

apenas no diagnóstico, terminar sempre com uma pergunta que leve o aluno ao próximo passo, e exemplos de estilo marcados como "de outros problemas — não copiar". É também aqui que vive a variável de ablação `MODO_ANOTACAO` (`inline` / `numerica` / `nenhuma`).

gerar_feedback(codigo, analise) e gerar_feedback_stream(codigo, analise)

As duas variantes da 2ª etapa (o único passo com LLM). A primeira retorna o feedback como JSON validado, com retentativas e backoff exponencial (usada no modo catálogo). A segunda é um gerador que entrega o texto token a token para o streaming SSE da API (efeito "digitando"), reutilizando exatamente o mesmo prompt. Ambas usam a temperatura de feedback (0.7).

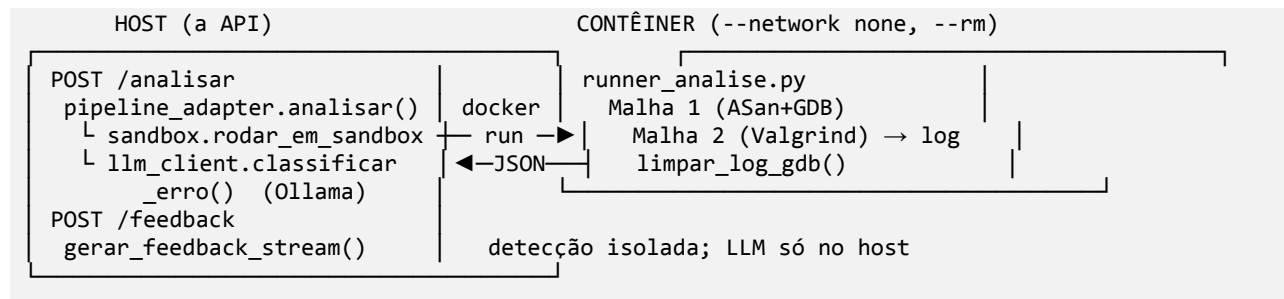
7. orquestrador.py — ponto de entrada (modo catálogo)

Percorre todos os `.c` da pasta de submissões e, para cada um, roda a cascata (Malha 1 → 2 → 3), limpa o log, chama `classificar_erro` e, no mesmo laço e em memória, chama `gerar_feedback`. Salva três artefatos por submissão com erro: uma linha no CSV (com 15 colunas, incluindo CWE, linhas do erro e o feedback), o código anotado (`<nome>_AnotacaoErro.c`) e o feedback (`feedback_<nome>.txt`). Distingue explicitamente erro de memória de erro de compilação, para o catálogo não rotular uma falha do gcc como bug de memória. A Malha 3 (perfilador) é carregada de forma protegida via `importlib` — se faltar, o pipeline de memória segue normalmente.

8. A API (pasta api/)

Expõe o mesmo pipeline como serviço para o CodeBench. A separação de responsabilidades é a chave da segurança: a **detecção** (que executa código não confiável) roda **isolada num contêiner Docker**; a **classificação e o feedback** (que precisam do Ollama) rodam **no host**, porque o contêiner sobe sem rede.

Diagrama do modo real



8.1. api/app.py — o serviço FastAPI

Define os endpoints e o formato de resposta. Pontos-chave:

- `/analisar` (POST JSON): roda a análise determinística e devolve o diagnóstico. Rápido.
- `/feedback` (streaming SSE): emite eventos em sequência — status (progresso imediato), diagnostico (o resultado determinístico), token* (os pedaços do texto do LLM) e fim.
- `_sem_internos()`: remove o campo `codigo_annotado` antes de responder ao cliente — devolvê-lo entregaria a localização exata do erro e furaria a proposta socrática.
- Variantes _arquivo:** `/analisar_arquivo` e `/feedback_arquivo` aceitam upload multipart, para testar sem embutir o código num JSON com `\n`. Há ainda `/saude` e `/teste` (página HTML de teste do SSE).

8.2. api/pipeline_adapter.py — a cola entre API e pipeline

Isola a API dos detalhes do pipeline: ela só chama `analisar()` e `stream_feedback()`. O adaptador decide entre **modo mock** (padrão, dados fixos para testar a API/SSE sem dependências) e **modo real** (`USAR_PIPELINE_REAL=1`), que chama a detecção no sandbox e, em seguida, a classificação/feedback do src/. No modo real, a detecção acontece no contêiner e a classificação/feedback no host — exatamente a divisão de segurança descrita acima.

8.3. api/sandbox.py — o wrapper de isolamento (Docker)

Função `rodar_em_sandbox(codigo, entrada, nome_arquivo)`. Escreve o código do aluno e o caso de teste num diretório temporário, monta esse diretório como somente-leitura no contêiner e dispara o `docker run` com as travas de segurança:

- `--network none`: sem rede (impede o código "ligar pra fora").
- `--read-only + --tmpfs /tmp`: sistema de arquivos imutável, exceto `/tmp` (onde as malhas compilam).
- `--memory 512m, --cpus 1, --pids-limit 128`: tetos de recursos (barram fork bomb e consumo excessivo do Valgrind).
- `--user 1000:1000`: roda como usuário sem privilégios (não-root).
- `--cap-add SYS_PTRACE + seccomp=unconfined`: libera apenas o `ptrace` e as syscalls que GDB/Valgrind precisam — mais restrito que `--privileged`.

Tem timeout por execução e um modo mock (`MODO_SANDBOX != docker`) que devolve um resultado de exemplo, deixando a API testável sem Docker instalado.

8.4. api/runner_analise.py — o "programa" dentro do contêiner

É o entrypoint da imagem. Recebe o caminho do .c montado em /work, roda a cascata de detecção (as mesmas Malhas 1 e 2 do src/, garantindo paridade com o orquestrador), limpa o log e imprime o resultado como JSON após um marcador (===RESULTADO_JSON===). O sandbox.py lê só o JSON após esse marcador, descartando o ruído dos prints das malhas. Lê a entrada autoritativa de /work/entrada.in (gravada pelo sandbox), sem depender da heurística de stdin.

8.5. api/Dockerfile.analise — a imagem de análise

Define a imagem: um Ubuntu 22.04 com gcc, gdb, valgrind, libasan8, libc6-dev e python3. Copia o pacote src/ e o runner_analise.py para dentro, cria um usuário sem privilégios e define /tmp como diretório de trabalho gravável. Detalhe importante: o build precisa ser feito **a partir da raiz do projeto** (docker build -f api/Dockerfile.analise -t analyse-mem:latest .), porque ele copia src/, que fica na raiz. Copiar o src/ inteiro mantém a imagem espelhando o código do pipeline (paridade total).

Fluxo da entrada (caso de teste) na API

A entrada enviada na requisição é **autoritativa**: o sandbox.py a grava em entrada.in no diretório montado, o runner_analise.py a lê de /work/entrada.in e a repassa às malhas como stdin (mesmo vazia). A heurística de adivinhar o stdin não é usada no caminho da API — a entrada vem sempre junto com o código. No uso offline (catálogo), sem entrada explícita, as malhas ainda recorrem ao .in/heurística.

9. Fundamentação teórica

- **FLAME (anotação inline)**: embasa a decisão de anotar o erro no código (`// @@`) em vez de citar o número da linha. O ganho vem da resolução de referência em contexto longo, não do tamanho do prompt.
- **Hellas et al. (vazamento de solução)**: mostra que LLMs entregam a solução ao aluno mesmo quando o prompt proíbe (até 65% de vazamento de solução-modelo). Daí as regras anti-vazamento reforçadas no prompt e a intenção de um filtro determinístico como etapa futura.
- **CWE / MITRE e Juliet Test Suite (NIST SARD)**: embasam o mapeamento determinístico do erro para um padrão citável e a curadoria da base de conhecimento por CWE.
- **RAG por chave exata**: recuperar o material de apoio pelo CWE (caso extremo de metadata filtering) evita o problema do distrator, amplificado sob a capacidade limitada de um modelo 7B.

10. Como rodar

Modo catálogo (offline)

```
python orquestrador.py
```

Saídas: `output/catalogo_erros_codebench.csv`, `output/codigos_annotados/` e `output/feedbacks/`. Requisitos: gcc, valgrind, gdb; Python 3 (requests, numpy, scipy); Ollama servindo `qwen2.5-coder:7b`.

Experimentos de ablação (sem editar código)

```
ANOTACAO_MODAL=inline    python orquestrador.py    # código anotado // @@ (padrão)
ANOTACAO_MODAL=numerica  python orquestrador.py    # original + linhas em texto
ANOTACAO_MODAL=nenhuma   python orquestrador.py    # baseline (sem dica de linha)
```

API

```
# modo mock (de dentro de api/)
uvicorn app:app --reload
```

```
# modo real (a partir da RAIZ do projeto)
docker build -f api/Dockerfile.analise -t analise-mem:latest .
USAR_PIPELINE_REAL=1 MODO_SANDBOX=docker uvicorn api.app:app --reload
```

11. Pendências e trabalhos futuros

- **Transporte do stream até o aluno**: como o SSE chega ao navegador depende de como o front do CodeBench recebe tempo real (SSE repassado? WebSocket? polling?) — a confirmar com a equipe.
- **Filtro determinístico anti-vazamento**: complementar às regras do prompt, robusto a conjugações verbais, como etapa pós-geração.
- **Malha 3 (perfilador)**: ainda exploratória; precisa de fundamentação teórica para compor oficialmente o pipeline.